https://dev.to/ibrahima92/advanced-typescript-types-cheat-sheet-with-examples-5414

Intersection Types

```
type LeftType = { id: number left: string }
type RightType = { id: number right: string }

type IntersectionType = LeftType & RightType

function showType(args: IntersectionType) {
console.log(args) }

showType({ id: 1, left: "test", right: "test" })
// Output: {id: 1, left: "test", right: "test"}
```

Union Types

```
type UnionType = string | number

function showType(arg: UnionType) { console.log(arg) }

showType("test")
// Output: test

showType(7)
// Output: 7
```

Generic Types

```
function showType<T>(args: T) { console.log(args) }

showType("test") // Output: "test"

showType(1) // Output: 1

interface GenericType<T> { id: number name: T }

function showType(args: GenericType<string>) {
console.log(args) }

showType({ id: 1, name: "test" }) // Output: {id: 1, name:
"test"}

function showTypeTwo(args: GenericType<number>) {
console.log(args) }

showTypeTwo({ id: 1, name: 4 })
// Output: {id: 1, name: 4}

interface GenericType<T, U> { id: T name: U }

function showType(args: GenericType<number, string>) {
console.log(args) }

showType({ id: 1, name: "test" })
// Output: {id: 1, name: "test"}

function showTypeTwo(args: GenericType<string,
string[]>) { console.log(args) }
```

Partial<T>

```
// all properties of the type T optional

interface PartialType { id: number firstName: string
lastName: string }

function showType(args: Partial<PartialType>) {
console.log(args) }

showType({ id: 1 }) // Output: {id: 1}

showType({ firstName: "John", lastName: "Doe" })
// Output: {firstName: "John", lastName: "Doe"}
```

https://dev.to/ibrahima92/advanced-typescript-types-cheat-sheet-with-examples-5414

showTypeTwo({ id: "001", name: ["This", "is", "a", "Test"] })
// Output: {id: "001", name: Array["This", "is", "a", "Test"]}
	## Required<T>
	// all properties of T required, even if prop is optional
	
	interface RequiredType {
	id: number firstName?: string lastName?: string
	}
	
	function showType(args: Required<RequiredType>) { console.log(args) }
	
	showType({ id: 1, firstName: "John", lastName: "Doe" })
	// Output: { id: 1, firstName: "John", lastName: "Doe" }
	
	showType({ id: 1 })
	// Error: Type '{ id: number: }' is missing the following properties from type 'Required<RequiredType>': firstName, lastName

https://dev.to/ibrahima92/advanced-typescript-types-cheat-sheet-with-examples-5414

Readonly<T>

```
interface ReadonlyType { id: number name: string }

function showType(args: Readonly<ReadonlyType>) {
  args.id = 4
  console.log(args)
}

showType({ id: 1, name: "Doe" })
// Error: Cannot assign to 'id' because it is a read-only property.
// Or:
interface ReadonlyType { readonly id: number name: string }
```

Pick<T, K>

```
 // create a new type from an existing model T by selecting some properties K of that type

interface PickType { id: number firstName: string lastName: string }

function showType(args: Pick<PickType, "firstName" | "lastName">) {
  console.log(args)
}
showType({ firstName: "John", lastName: "Doe" })
// Output: {firstName: "John"}

showType({ id: 3 })
// Error: Object literal may only specify known properties, and 'id' does not exist in type 'Pick<PickType, "firstName" | "lastName">
```

Omit<T, K>

```
// remove K properties from the type T

interface PickType { id: number firstName: string lastName: string }

function showType(args: Omit<PickType, "firstName" | "lastName">) {
  console.log(args)
}

showType({ id: 7 })
// Output: {id: 7}

showType({ firstName: "John" })
// Error: Object literal may only specify known properties, and 'firstName' does not exist in type 'Pick<PickType, "id">'
```

Extract<T, U>

```
// Extract from T all properties that are assignable to U

interface FirstType { id: number firstName: string lastName: string }

interface SecondType { id: number address: string city: string }

type ExtractType = Extract<keyof FirstType, keyof SecondType>
// Output: "id"
```

https://dev.to/ibrahima92/advanced-typescript-types-cheat-sheet-with-examples-5414

Exclude<T, U>

```
// construct a type by excluding properties that are already present in two different types

interface FirstType { id: number firstName: string lastName: string }

interface SecondType { id: number address: string city: string }

type ExcludeType = Exclude<keyof FirstType, keyof SecondType>

// Output; "firstName" | "lastName"
```

Record<K,T>

```
// set of properties K of a given type T

interface EmployeeType { id: number fullname: string role: string }

let employees: Record<number, EmployeeType> = {
  0: { id: 1, fullname: "John Doe", role: "Designer" },
  1: { id: 2, fullname: "Ibrahima Fall", role: "Developer" },
  2: { id: 3, fullname: "Sara Duckson", role: "Developer" },
}

// 0: { id: 1, fullname: "John Doe", role: "Designer" },
// 1: { id: 2, fullname: "Ibrahima Fall", role: "Developer" },
// 2: { id: 3, fullname: "Sara Duckson", role: "Developer" }
```

NonNullable<T>

```
// remove null and undefined from the type T.

type NonNullableType = string | number | null | undefined

function showType(args: NonNullable<NonNullableType>) { console.log(args) }

showType("test")
// Output: "test"

showType(1)
// Output: 1

showType(null)
// Error: Argument of type 'null' is not assignable to parameter of type 'string | number'.

showType(undefined)
// Error: Argument of type 'undefined' is not assignable to parameter of type 'string | number'.
```

Mapped Types

```
// take an existing model and transform each of its properties into a new type

type StringMap<T> = { [P in keyof T]: string }

function showType(arg: StringMap<{ id: number; name: string }>) { console.log(arg) }

showType({ id: 1, name: "Test" })
// Error: Type 'number' is not assignable to type 'string'.

showType({ id: "testId", name: "This is a Test" })
// Output: {id: "testId", name: "This is a Test"}
```